

Unit-VI

Applications of NLP

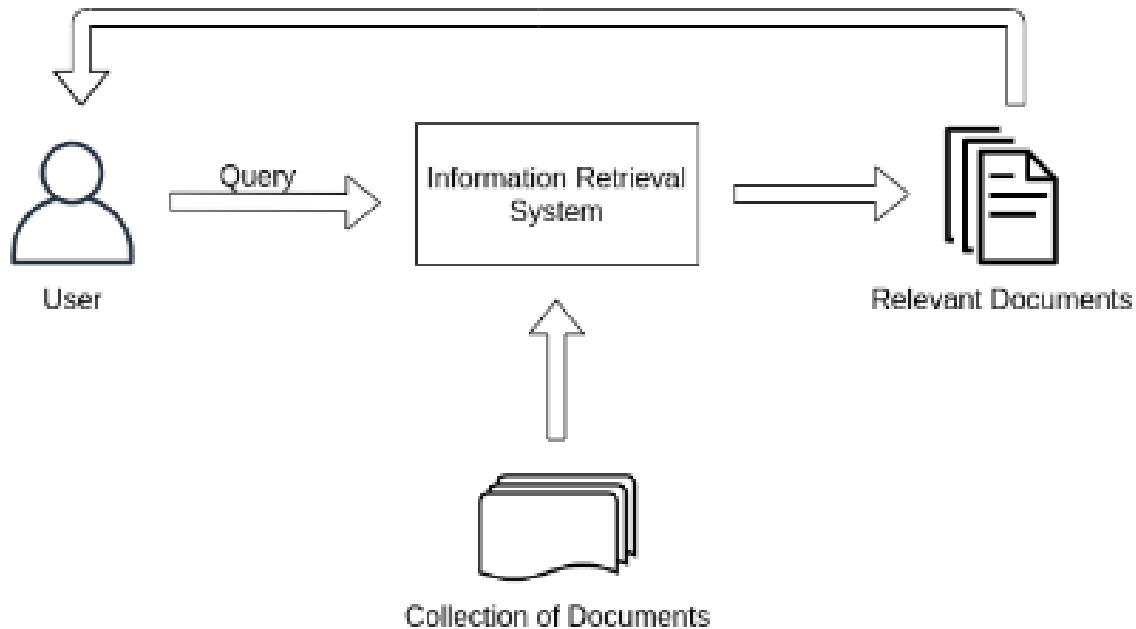
Syllabus Outline

- Information retrieval-Vector Space Model
- Information Extraction using sequence labelling
- Question answers system
- Categorization
- summarization
- sentiment analysis
- Named Entity Recognition.
- Analyzing text with NLTK
- Chatbot using Dialogflow



Information Retrieval

“Information Retrieval is the process of finding desired documents from a collection of documents.”



Vector Space Model

- The Vector Space Model (VSM) is a mathematical framework used in information retrieval and natural language processing (NLP) to represent and analyze textual data.

The various applications are:

- Text Mining
- Document Retrieval
- Text-based Machine Learning Tasks Like Document Classification
- Information Retrieval
- Text Similarity Analysis
- Chatbots



Information Extraction



Machine Translation



Chatbots

Vector Space Model

- Vector space models are algebraic models that are often used to represent text (although they can represent any object) as a vector of identifiers.
- With these models, we are able to identify whether various texts are similar in meaning, regardless of whether they share the same words.

Where are you from?
Where are you going?



Same words,
Different meaning

What is your age?
How old are you?



Different words,
Same meaning

Why Learn Vector Space Model?

Vector space models will also allow you to capture dependencies between words. In the following two examples, you can see the word “cereal” and the word “bowl” are related. Similarly, you can see “sell” depends on “buy”. With vector space models, you will be able to capture this and many other types of relationships among different sets of words.

- You eat *cereal* from a *bowl*
- You *buy* something and someone else *sells* it

Why learn vector space models?

Where are you heading?
Where are you from?



Different meaning

What is your age?

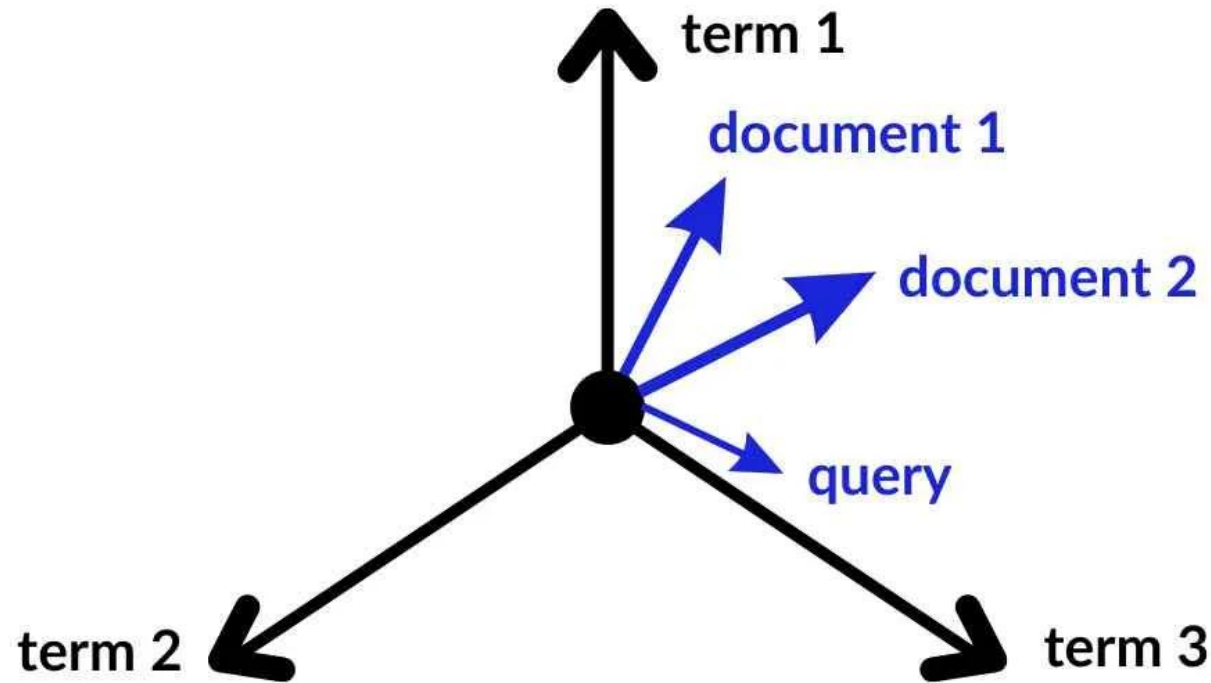
How old are you?



Same Meaning

Vector Space Model

- The Vector Space Model represents documents and terms as vectors in a multi-dimensional space. Each dimension corresponds to a unique term in the entire corpus of documents.



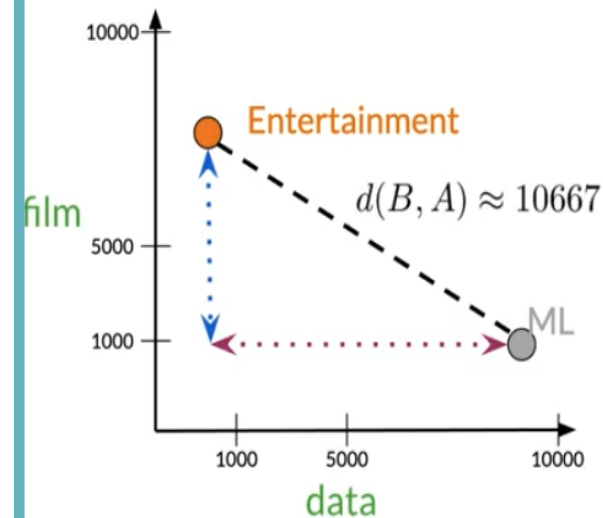
Each dimension corresponds to a unique term, while the documents and queries can be represented as a vector within that space.

Euclidean Distance: Measuring Textual Proximity

- One fundamental concept in VSMs is the Euclidean distance.
 - It's the “straight line” distance between two points in vector space.
 - Let us assume that you want to compute the distance between two points: A,B.
- To do so, you can use the euclidean distance defined as

$$d(B, A) = \sqrt{((B_1 - A_1)^2 + (B_2 - A_2)^2)}$$

Euclidean distance



Corpus A: (500,7000)



Corpus B: (9320,1000)

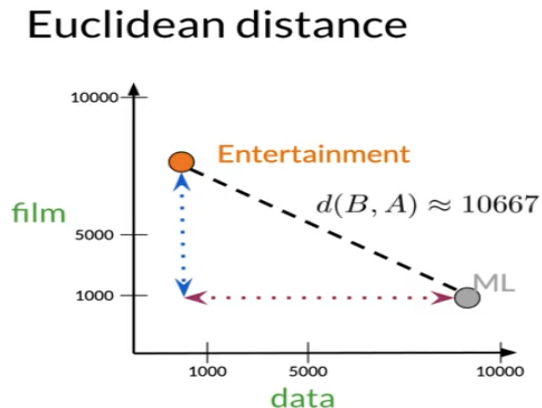
$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$
$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(8820)^2 + (-6000)^2}$$

Vector Space Model

Euclidean Distance for N-Dimensional Vectors

- The approach to calculate euclidean distance for n-dimensional vectors is similar to two dimension calculation, shown in the example above.
- First get the difference between each of the dimensions.
- Square the differences, then sum them up and then get the square root of the results.
- The formula is just the norm of the difference between the vectors that we are comparing.



Corpus A: (500,7000)

Corpus B: (9320,1000)

$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$
$$c^2 = a^2 + b^2$$
$$d(B, A) = \sqrt{(8820)^2 + (-6000)^2}$$

Vector Space Model

Euclidean distance for n-dimensional vectors

		$\vec{w}$	$\vec{v}$
	data	boba	ice-cream
AI	6	0	1
drinks	0	4	6
food	0	6	8

$$\begin{aligned} &= \sqrt{(1-0)^2 + (6-4)^2 + (8-6)^2} \\ &= \sqrt{1+4+4} = \sqrt{9} = 3 \end{aligned}$$

$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^n (v_i - w_i)^2} \longrightarrow \text{Norm of } (\vec{v} - \vec{w})$$

- The Euclidean distance is a straight line between points

- To calculate the euclidean distance, we calculate the norm of the difference between the vectors that we are comparing.

- By using this metric, we can get a sense of how similar two documents or words are.

Vector Space Model

Cosine Similarity

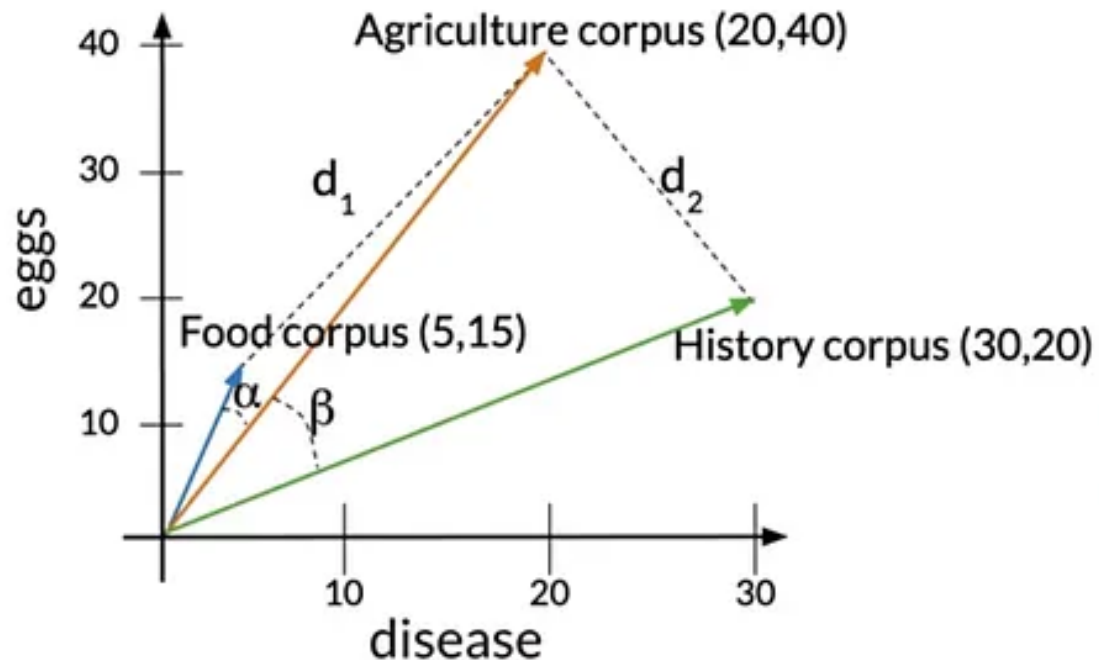
- One of the issues with euclidean distance is that it is not always accurate and sometimes we are not looking for that type of similarity metric.
- For example, when comparing large documents to smaller ones with euclidean distance, one could get an inaccurate result.
- In the following example, the word totals in the corpora differ from one another.
- For example, agriculture and history corpus have a similar number of words, while the food corpus has a relatively small number. If we use Euclidean Distance as a metric to find similarity, we would say agriculture is similar to history, but not to food.
- Normally, agriculture and food are more similar than agriculture and history. However the food corpus is much smaller than the agriculture corpus. To further clarify, although the history corpus and the agriculture corpus are different, they have a smaller euclidean distance.

Vector Space Model

Cosine Similarity

To solve this problem, we look at the cosine between the vectors. The main advantage of the cosine similarity metric over the euclidean distance is that it isn't biased by the size difference between the representations.

Euclidean distance vs Cosine similarity



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle
between the vectors

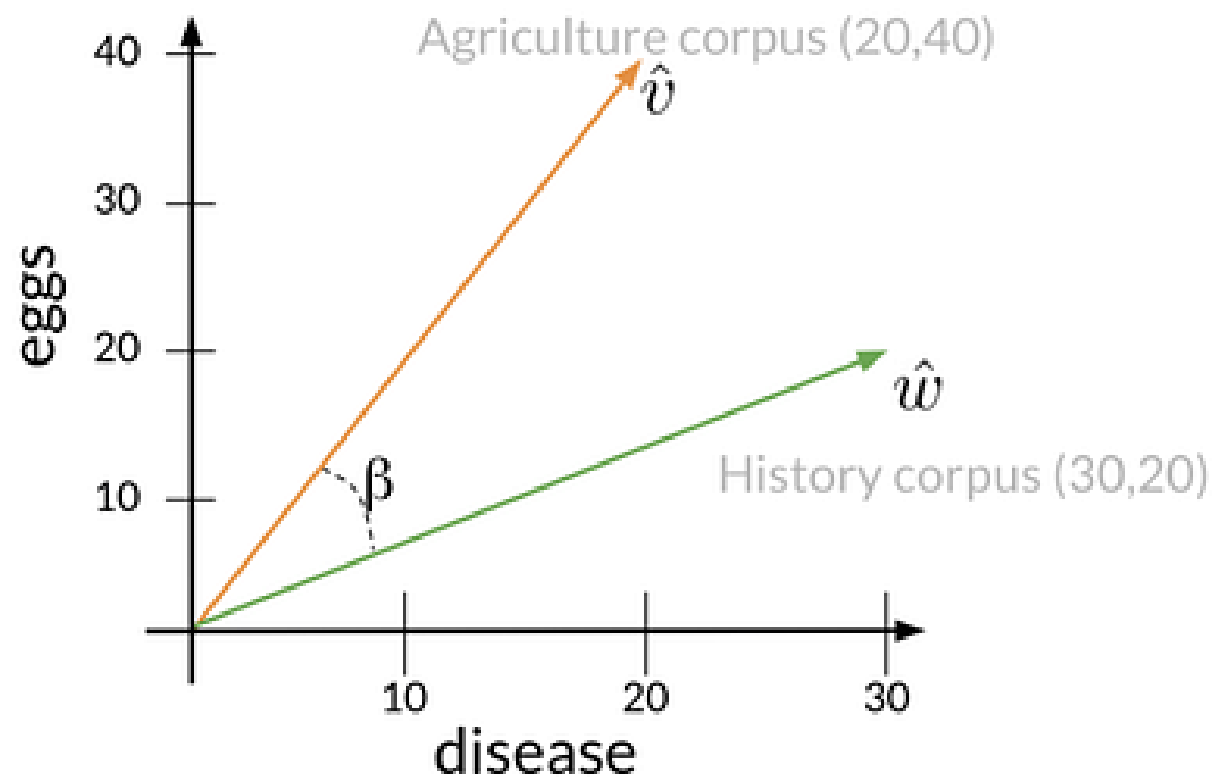
Vector Space Model

Cosine Similarity

- Cosine Similarity Formula
- Before we get into cosine similarity, there two functions we want to review:
- The norm of a vector is defined as: $\|\vec{v}\| = \sqrt{\sum_{i=1}^n |v_i|^2}$
- The dot product is defined as: $\vec{v} \cdot \vec{w} = \sum_{i=1}^n v_i \cdot w_i$
- The cosine similarity equation is then: $\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$
-

Vector Space Model

Cosine Similarity



$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

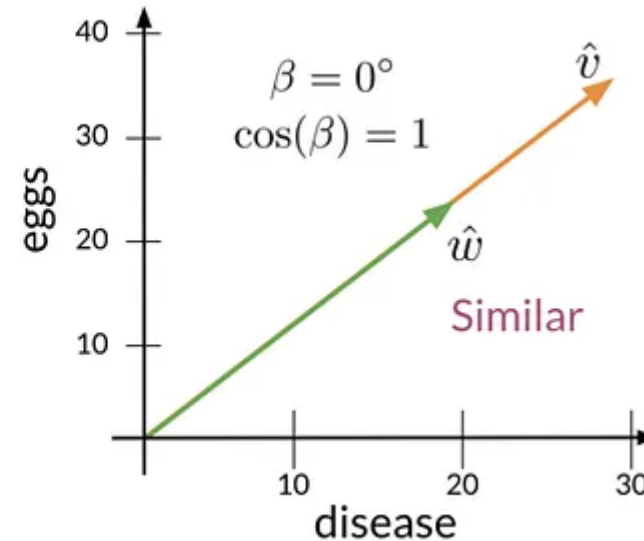
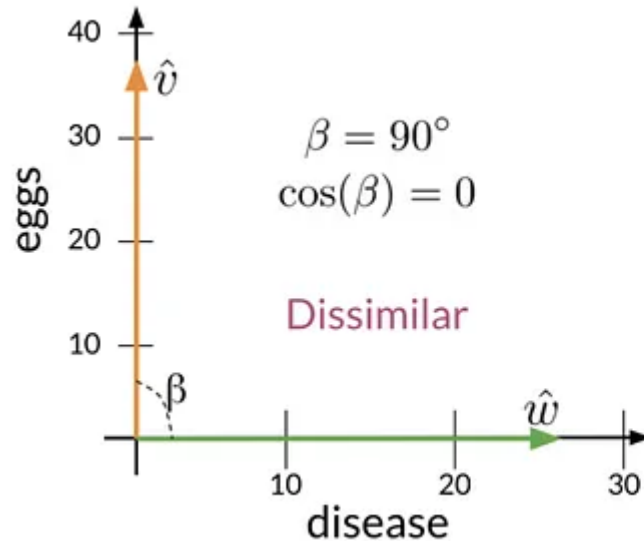
$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}} = 0.87$$

Vector Space Model

Cosine Similarity

- Cosine Similarity Interpretation:
- The more similar two vectors are, the more closer to 1
- If two vectors are orthogonal, the cosine is 0



Vector Space Model example

- Let's walk through a simple example of the Vector Space Model (VSM) using a small corpus of documents and a query.
- In this example, we'll represent documents and a query as vectors and calculate cosine similarity to retrieve relevant documents based on the query.
- **Step 1: Corpus and Query**
- Let's start with a small corpus of three documents and a query:
- **Document 1:** "The quick brown fox jumps over the lazy dog."
Document 2: "A brown dog chased the fox."
Document 3: "The dog is lazy."
- **Query:** "brown dog"

Vector Space Model example

- Step 2: Create the Document-Term Matrix (DTM)
- We create a DTM where rows represent documents and columns represent terms. We'll use TF-IDF values for each term in the matrix:

	a	brown	chased	dog	fox	is	jumps	lazy	over	quick	the
Doc 1	0	0.29	0	0.29	0.29	0	0.29	0.29	0.29	0.29	0.58
Doc 2	0.41	0.29	0.41	0.29	0.29	0	0	0	0	0	0.41
Doc 3	0	0	0	0.41	0	0.41	0	0.41	0	0	0.41
Query	0	0.71	0	0.71	0	0	0	0	0	0	0

- Here, we've calculated TF-IDF values for each term in the documents and the query. You can use different formulas for TF-IDF, but this is common.

Vector Space Model example

Step 3: Vectorize the Query

The query is also represented as a vector. In this case, it's a simple binary vector where 1 represents the presence of a term and 0 represents the absence:

		a		brown		chased		dog		fox		is		jumps		lazy		over		quick		the		
		-----		---		-----		-----		---		---		---		-----		-----		-----		---		
	Query		0		1		0		1		0		0		0		0		0		0		0	

Step 4: Calculate Cosine Similarity

Now, we calculate the cosine similarity between the query vector and each document vector. The formula for cosine similarity is:

$$\text{Cosine Similarity} = \frac{\text{Query Vector} \cdot \text{Document Vector}}{\|\text{Query Vector}\| \cdot \|\text{Document Vector}\|}$$

Vector Space Model example



Using this formula, we calculate the cosine similarity between the query and each document:

Cosine Similarity(Query, Doc 1) $\approx$ 0.58

Cosine Similarity(Query, Doc 2) $\approx$ 0.29

Cosine Similarity(Query, Doc 3) $\approx$ 0.41

Step 5: Rank Documents by Similarity

The documents are ranked by their cosine similarity values in descending order:

Document 1: Cosine Similarity $\approx$ 0.58

Document 3: Cosine Similarity $\approx$ 0.41

Document 2: Cosine Similarity $\approx$ 0.29

So, based on cosine similarity, Document 1 is the most relevant to the query “brown dog,” followed by Document 3 and then Document 2. This demonstrates how the Vector Space Model can be used for information retrieval and ranking documents based on their similarity to a query.

• Information Extraction using sequence

Labelling

- Information extraction is an approach for identifying and retrieving specific information from unstructured data sources
- Sequence labeling is a fundamental technique in NLP that is used to identify and label the components of a sequence, such as words or phrases in a sentence.
- It is often used as a preprocessing step for other NLP tasks, such as part-of-speech tagging or named entity recognition, and is a key component of many NLP applications.

• **Information Extraction using sequence Labelling Applications :**

- It can be used in **text classification** to categorize documents and in **sentiment analysis** to determine the sentiment of a text.
- In **information retrieval**, it helps to clarify the context and meaning of a query.
- **Machine translation** to identify the grammatical structure of a sentence and to facilitate the translation process.

• **Information Extraction using sequence Labelling**



Main Tasks of Sequence Labeling

- 1) Part-Of-Speech Tagging
- 2) Named Entity Recognition
- 3) Chunking
- 4) Semantic Role Labeling

• Information Extraction using sequence Labelling

1) Part-Of-Speech Tagging

- Part-of-speech (POS) tagging is the process of labeling the parts of speech (such as *nouns*, *verbs*, and *adjectives*) in a sentence.
- In information retrieval, it can help the search engine to provide more relevant and accurate results by distinguishing different parts of speech.

• **Information Extraction using sequence Labelling**

2) Named Entity Recognition

- Named entity recognition (NER) is the task of identifying and classifying named entities (such as people, organizations, and locations) in text.
- As POS tagging, it can be used to extract information from a large corpus of texts and help identify more quickly what is wanted.

• **Information Extraction using sequence Labelling**



3) Chunking

- Chunking is a task of sequence labeling that involves dividing a sequence of words into chunks or non-overlapping sub-sequences.
- These chunks are typically tagged with a label that indicates their type or role in the sequence.
- It is often used as a preprocessing step for other natural language processing tasks, such as named entity recognition or part-of-speech tagging.

• Information Extraction using sequence Labelling



4) Chunking

- Suppose we have the following sentence: “The quick brown fox jumps over the lazy dog.”
- Using chunking, we might divide the sentence into the following chunks:
 - “The quick brown fox” (noun phrase)
 - “jumps” (verb)
 - “over the lazy dog” (noun phrase)
- This is one way to chunk by identifying the noun phrases in the sentence.
- The resulting sequence of chunks and tags would be:

(NP, The quick brown fox) (V, jumps) (NP, over the lazy dog)

• Information Extraction using sequence Labelling



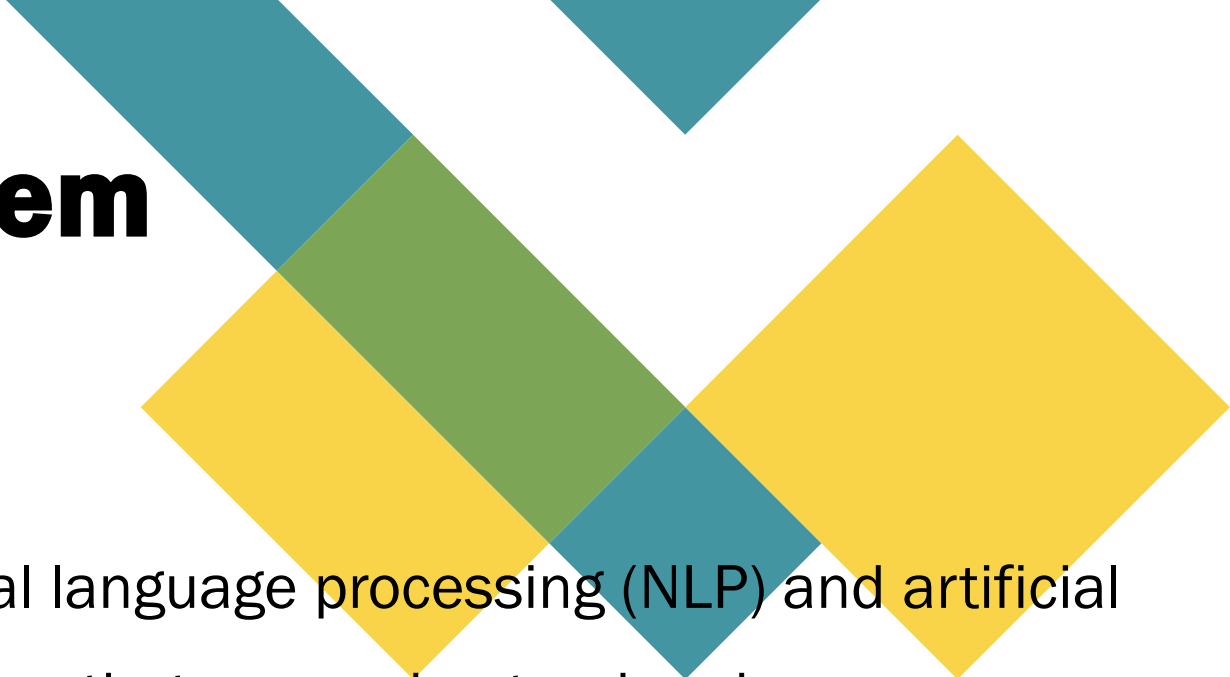
5) Semantic Role Labeling

Semantic Role Labeling (SR) goes beyond identifying the grammatical role of words as Part-Of-Speech Tagging but focuses on determining their meaning and the relationships between them.

It typically involves the following steps :

1. Identifying the **predicate** in a sentence
2. Identifying the **arguments** of the **predicate**
3. Labeling the arguments with their **corresponding roles**

Question Answers System



- Question answering (QA) is a field of natural language processing (NLP) and artificial intelligence (AI) that aims to develop systems that can understand and answer questions posed in natural language.
- Unlike traditional keyword searches, a Natural Language Question Answering system does not return a complete document to the user.
- Instead, users ask a question in natural language and receive a specific answer in return.

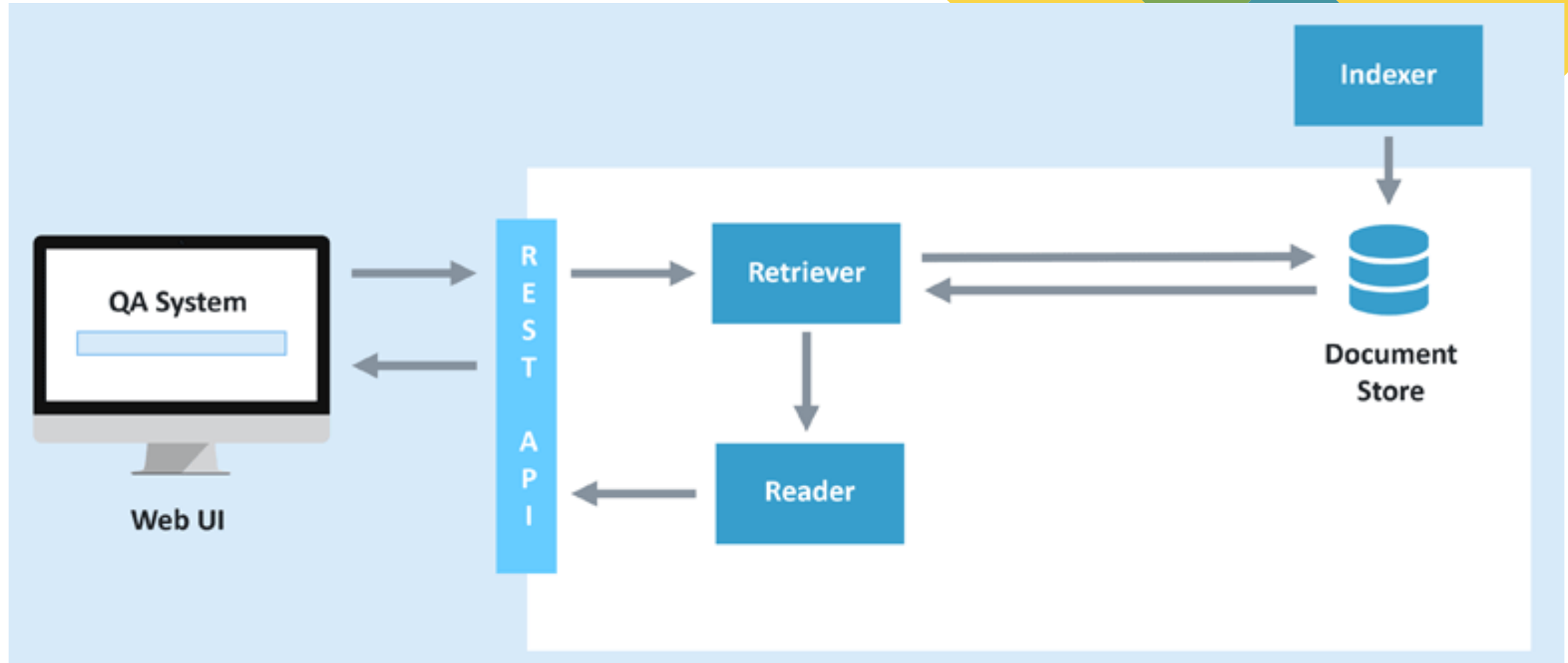
Question Answers System

How does a natural language question answering system work?

- Developing question answering systems has been a hot topic in IT for quite some time.
- Earlier, there were attempts to set up complex rules to enable a system to understand a user's naturally worded question and to provide an answer.
- The natural language question answering systems of today often take an **extractive** approach, and consist of a **retriever** and a **reader**.
- The answers to the questions are not stored in large databases, rather the system attempts to find and extract an appropriate answer to a user's question from a mass of texts.
- Firstly, documents relevant to the user's question are loaded from a document store.
- Then, the reader attempts to extract the answer to the user's question.

Question Answers System

How does a natural language question answering system work?



Question Answers System



- **What is a document store?**
- The document store is responsible for providing relevant documents.
- There are different ways of doing this.
- Often, a simple or reverse index is used.
- One well-known, commonly used such reverse index is Apache Lucene, which is used as an index in Elasticsearch. It's a fast, efficient way to call up documents.

Question Answers System



-
- **What is a retriever?**
 - A retriever is responsible for retrieving relevant documents for the user's question. First of all, it attempts to **extract the relevant terms in the question**.
 - It then uses these to retrieve relevant documents.
 - In order to turn a user's question into the type of query a retriever can process, various Natural Language Processing (NLP) techniques are used.

- These include:

i) **Removing punctuation**

Full stops, commas and other punctuation are superfluous in retrieving relevant documents. These are therefore removed from the user's question.

Question Answers System



ii) Removing stopwords

Stopwords are commonly occurring words which don't have a significant impact on content. Examples include articles such as 'the', 'a', 'an'. These words are therefore filtered out.

iii) Tagging entities

Entities, such as products or names, are usually very relevant to the query. These are therefore incorporated into the query.

iv) Stemming

Words can appear in different forms or conjugations (walk, walked, walking, etc.). As they may well appear in different forms within a document, such words are reduced to a base form before being incorporated into the query.

Question Answers System

- These steps help create a query, which is then made to the document store. The retriever grabs the most relevant documents and passes these on to the reader to extract an answer to the user's question.

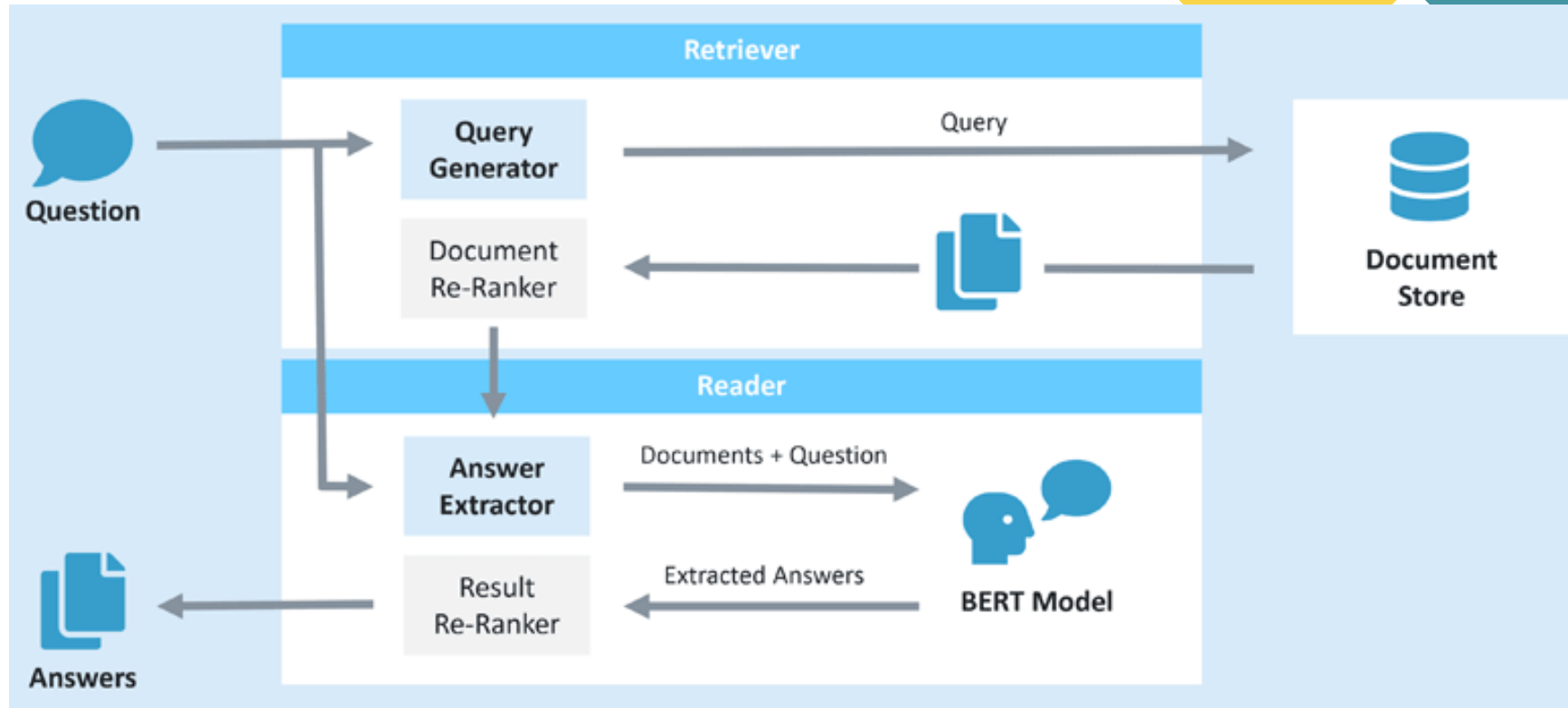


Figure 3: The BERT model in a natural learning question answering system

Question Answers System



- What is a reader in a question answering system?
- A reader is responsible for extracting an answer from the documents it receives. Using a suitable language model, it tries to understand both the question and the documents and to extract the most appropriate answer from the texts.
- What is a language model?
- A language model is used to extract answers from the texts submitted. One such model is the frequently used **BERT model** illustrated in Figure 2, however other language models are also available. A language model has been trained to **calculate the probability of a word or phrase occurring**.
- There are several different types of language model. The quality of their results depends on the areas in which they are employed. Many of these models are take a *transformer* or *attention* based approach which enables them to comprehend and process language.

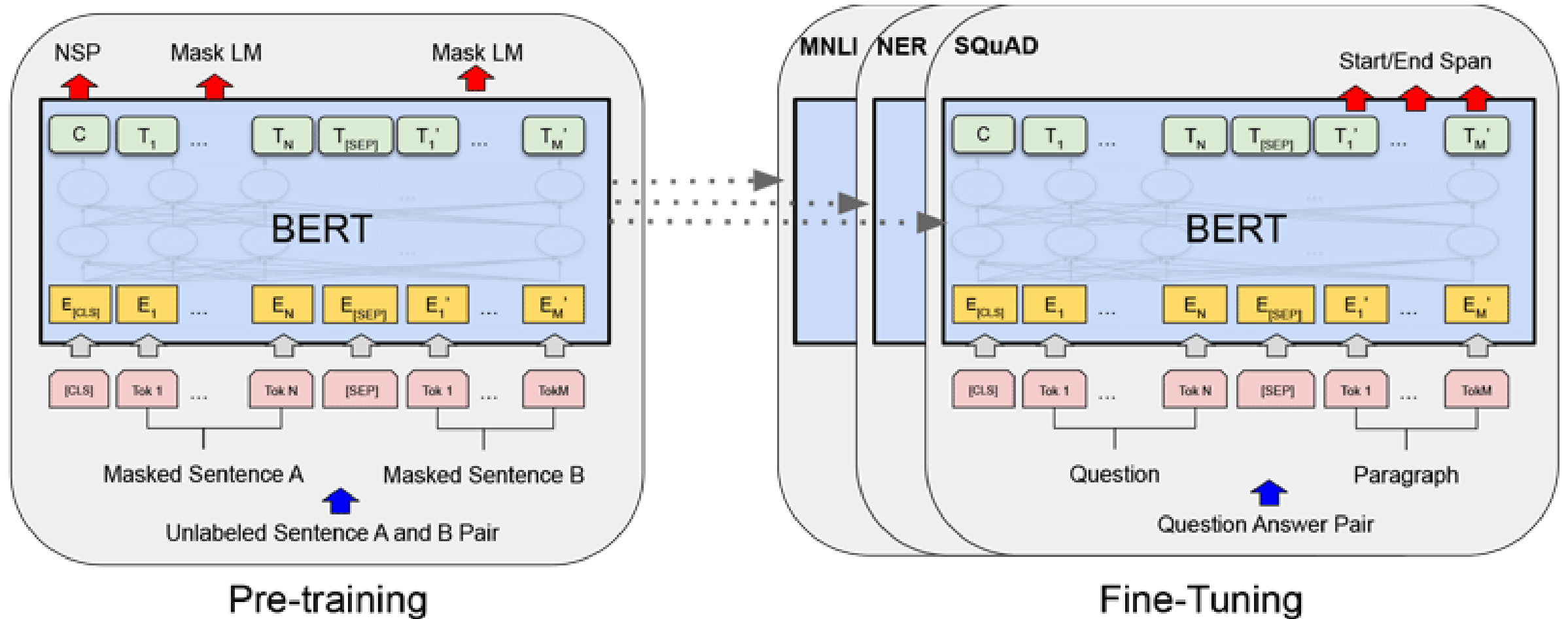
How does the BERT model work?

- Devlin et al's Bidirectional Encoder Representations from Transformers, commonly known as BERT, is a powerful language model.
- It builds on the encoder from Vaswani et al's *Transformers* and *attention* mechanism.
- It starts by dividing the input into individual words, known as tokens.
- These are augmented with further BERT-specific tokens and entered into the model. For a Question Answering scenario, the input consists of both the question and the paragraph from which the answer is to be extracted.
- There are multiple layers of transformers between the input and output of the model, which are used to compute the answer.

How does the BERT model work?

- The model outputs start and end tokens to determine the best answer. These are then used to help return an answer to the user.
-  At this stage, the model is not yet able to compute meaningful answers itself. To make this possible, the model needs to be trained. Initially, the model was trained to have general reading comprehension. This enables it to be used in further areas.
- To this end, the BERT model was pre-trained using the over 800 million words in BooksCorpus, as well as the English version of Wikipedia. For the latter, the model was only trained on the main body of the individual articles, encompassing over 2.5 billion words.
- The model was able to learn which words or phrases are dependent on or commonly used with which others, and can use this knowledge for other tasks. The advantage of pre-training a model is that it is later relatively easy to adapt it to a range of specific tasks – such as finding answers in paragraphs without necessarily needing to re-train it.

How does the BERT model work?



How the BERT model works for Natural Language Question Answering^[2]

How does the BERT model work?



-
- The pre-trained model was further prepared for its extractive question answering tasks using the SQuAD data set.
 - The SQuAD data set consists of 100,000 pairs of questions and answers to selected Wikipedia articles. By working through these sample pairs, the model builds on its previously learned general language comprehension and learns to extract and return the right answers to the questions.
 - A pre-trained BERT model can be directly employed in a Question Answering System. However, by following up by using data sets from specialist interest areas or certain types of document, the model can be trained to deliver even better results in a certain area.

Categorization



- Categorization in text mining means sorting documents into groups.
- Automatic document classification uses a combination of natural language processing (NLP) and machine learning to categorize customer reviews, support tickets, or any other type of text document based on their contents.
- Note: In this context, we use the terms document categorization and document classification synonymously.

Categorization

- NLP-based document categorization enables you to sort through huge numbers of documents without actually reading them yourself.
- Much like a librarian places a book about horses on a shelf labeled “Animals,” an NLP classifier can categorize a product description as “Sauces” because the text mentions ketchup or mustard. And because text classifiers work at the speed of computers, they can categorize thousands of documents in less time than it takes to drink your morning coffee.
- Rather than painstakingly sorting through them by hand, you can immediately find and focus on the documents you care about.

Categorization

Four Methods of Document Categorization

-
- Lexalytics supports four methods of document categorization.
 - Query Topics.
 - Machine learning models
 - Auto-Categories
 - Concept Topics

Categorization

Four Methods of Document Categorization

- Lexalytics supports four methods of document categorization.

1. Query Topics use Boolean operators to decide whether a document belongs in a given category by looking for the presence or absence of key words and phrases. Queries are simple to understand, easy to set up, and work best when you know that a particular set of words are useful as categorization markers. But the more meanings a word has, the more layers you need to add to your query topic. This can become cumbersome, unreliable and unsustainable.

Categorization

Four Methods of Document Categorization

2. Machine learning models categorize documents by identifying statistical connections between words and phrases and human-provided examples of where documents should be sorted. In effect, text classification algorithms, including Support Vector Machines (SVM) and Naive Bayes, learn the likelihood that a given set of words are related to a particular category. Model-based classifiers are efficient and require less human input than queries. But models require a training set of data to learn from, and, without maintenance, will quickly grow obsolete.

3. Auto-Categories use the Lexalytics Concept Matrix™ to compare your documents to 400 first-level categories and 4,000 second-level categories based on Wikipedia's own taxonomy. Auto-categories work out of the box, requiring no customization at all. They're the easiest tool to use in our categorization toolbox but cannot be changed or tuned.

Categorization

Four Methods of Document Categorization



4. Concept Topics use the Concept Matrix to broaden the scope of a user-defined category beyond a few example keywords. When you create a new Concept Topic, you're asked to give 4-6 examples of words that fall into that category. Then we use the Concept Matrix to find other words that are semantically related to the larger idea.

- Auto-Categories and Concept Topics are good at sorting content into broader, idea-based categories but don't work as well on shorter documents.

Summarization

- Consider a scenario wherein instead of reading an entire article or research paper you could just read the most important statements, this is possible through text summarization, a widely used technique in NLP.
- Text summarization takes an input of a sequence of words aka the input article and returns an output of words aka the summary, making it an essential application of sequence-to-sequence models in NLP machine learning.
- Text summarization can be a useful case study in domains like
 - **Financial Research**
 - **Question-answer Bots**
 - **Media Monitoring**
 - **Social Media Marketing**

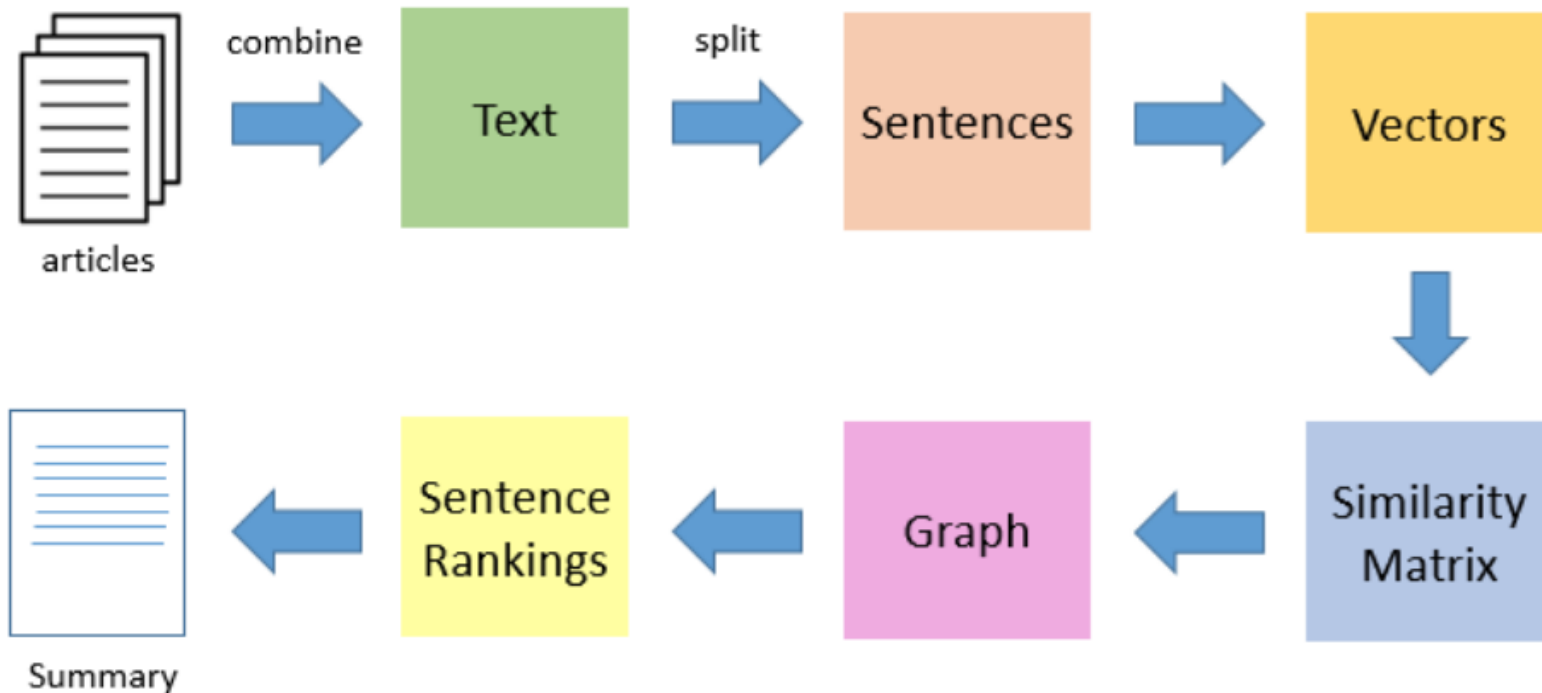
Summarization



- “Automatic text summarization is the task of producing a concise and fluent summary while preserving key information content and overall meaning”
- Text summarization refers to the technique of condensing a lengthy text document into a succinct and well-written summary that captures the essential information and main ideas of the original text, achieved by highlighting the significant points of the document.

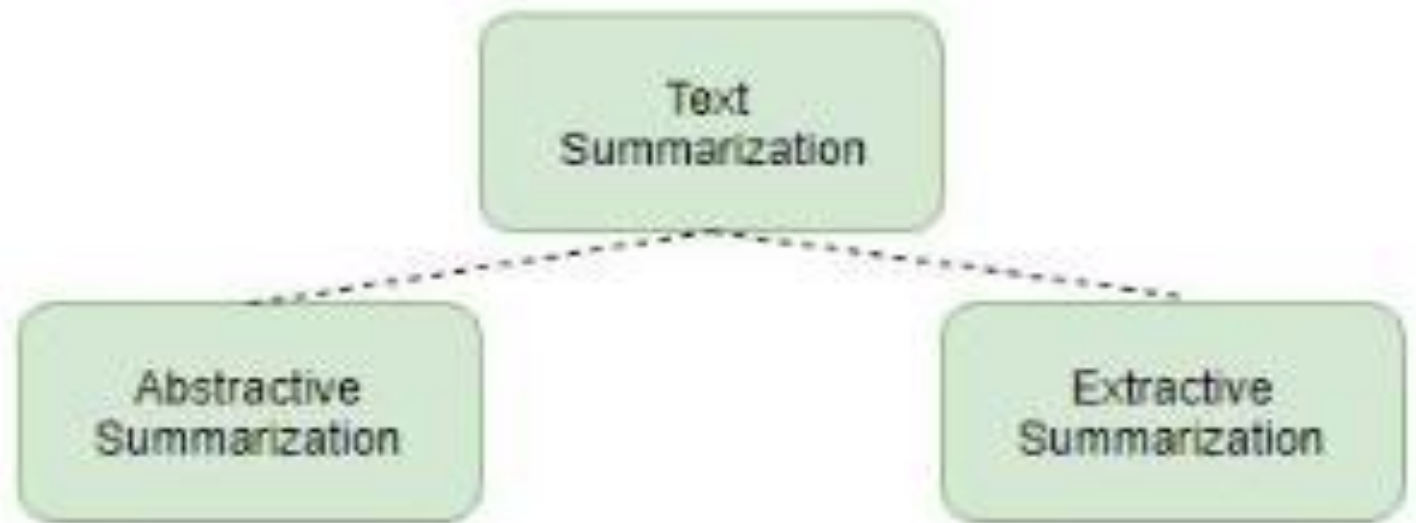
Summarization

- In this approach we build algorithms or programs which will reduce the text size and create a summary of our text data. This is called automatic text summarization in machine learning.
- Text summarization is the process of creating shorter text without removing the semantic structure of text.



Summarization

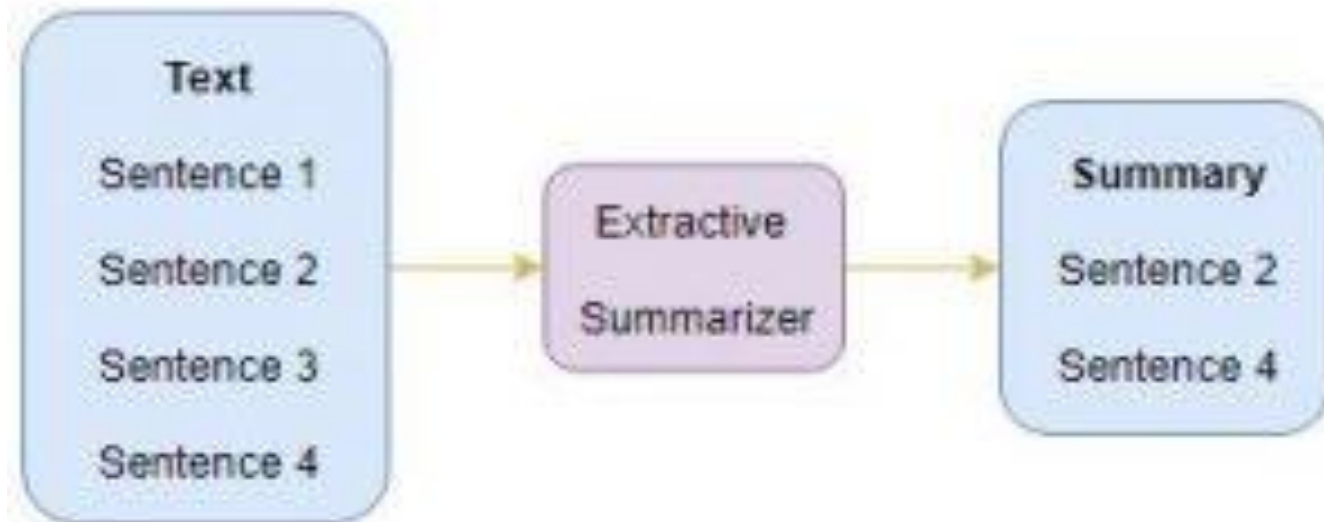
- There are broadly two different approaches that are used for text summarization:
- **Extractive Summarization**
- **Abstractive Summarization**



Summarization

1) Extractive Summarization

- The name gives away what this approach does. **We identify the important sentences or phrases from the original text and extract only those from the text.** Those extracted sentences would be our summary. The below diagram illustrates extractive summarization:

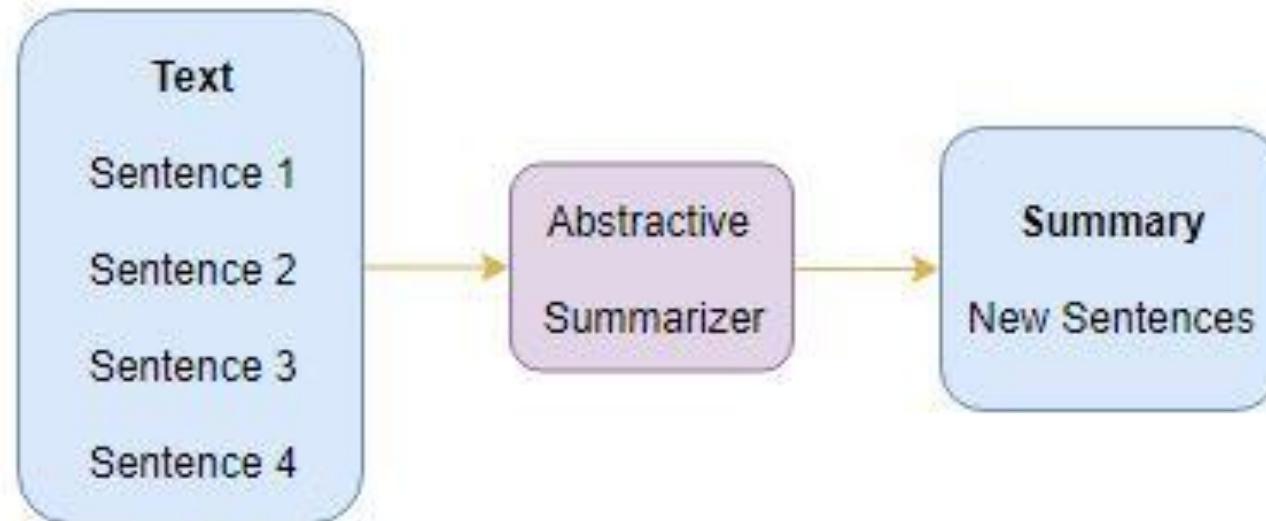


Summarization

2) Abstractive Summarization

- This is a very interesting approach. Here, we generate new sentences from the original text. This is in contrast to the extractive approach we saw earlier where we used only the sentences that were present. The sentences generated through abstractive summarization might not be present in the original text:

•



Named Entity Recognition.

- Named entity recognition, or NER, is a process that extracts information from text.
It's also referred to as entity chunking, entity extraction, or entity identification.
- The goal is to identify, sort, and rank pieces of information by importance.
- Breaking this term down into two parts can help us better understand it:
- **Named Entity:** A named entity is any object that can be referenced *by name* in text.
- **Recognition:** NER systems are trained to *recognize* these objects and sort them into helpful classifications called entity types.

Named Entity Recognition.

- NER essentially extracts and categorizes the detected entity into a predetermined category.
- The category can be generic like Organization, Person, Location, Time, etc., or a custom category depending on the use case such as Healthcare Terms, Programming Language, etc.
- For example, an NER model detects “football” as an entity in a paragraph and classifies it into the category of sports.

I hear ^{Place} **Berlin** is wonderful in the ^{Time} **winter**

In the image above, Berlin and winter are two entities and are arranged into categories of place and time, respectively.

Named Entity Recognition.

- **Key concepts of NER**

- Named entities are not the only concept to understand within the NER world. Several other terms should be explained to understand the topic better.

1) **POS tagging.** Standing for "part-of-speech tagging," this process assigns labels to words in a text corresponding to their specific part of speech, such as adjectives, verbs, or nouns.

2) **Corpus.** This is a collection of texts used for linguistic analysis and training NER models. A corpus can range from a set of news articles to academic journals or even social media posts.

Named Entity Recognition.

- Key concepts of NER

3) **Chunking.** This is an NLP technique that groups individual words or phrases into "chunks" based on their syntactic roles, creating meaningful clusters like noun phrases or verb phrases.

4) **Word embeddings.** These are dense vector representations of words, capturing their semantic meanings. Word embeddings translate words or phrases into numerical vectors of fixed size, making it easier for machine learning models to process. Tools like Word2Vec and GloVe are popular for generating such embeddings, and they help in understanding the context and relationships between words in a text.

Methods of NER(Named Entity Recognition)



1. Dictionary-based
2. Rule-based
3. Machine learning-based
4. Deep learning-based

Methods of NER(Named Entity Recognition)

1. Dictionary-based

- This is the simplest NER method. In this approach, a dictionary containing vocabulary is used. Basic string-matching algorithms check whether the entity is present in the given text against the items in the vocabulary.
- This method is generally not employed because the dictionary that is used is required to be updated and maintained consistently.

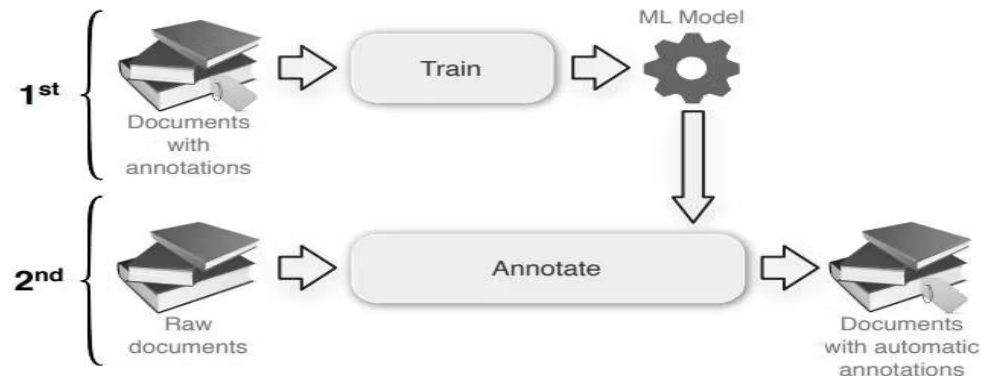
2. Rule-based

- In this method, a predefined set of rules for information extraction is used which are pattern-based and context-based.
- Pattern-based rule uses the morphological pattern of the words while context-based uses the context of the word given in the text document.

Methods of NER(Named Entity Recognition)

3. Machine learning-based

- This method solves a lot of limitations of the above two methods. It is a statistical-based model that tries to make a feature-based representation of the observed data. It can recognize an existing entity name even with small spelling variations.
- The machine learning-based approach involves two phases for doing NER. The first phase trains the ML model on annotated documents.
- In the next phase, the trained model is used to annotate the raw documents. The process is similar to a normal ML model pipeline.



Methods of NER(Named Entity Recognition)



- **Deep Learning Approaches**

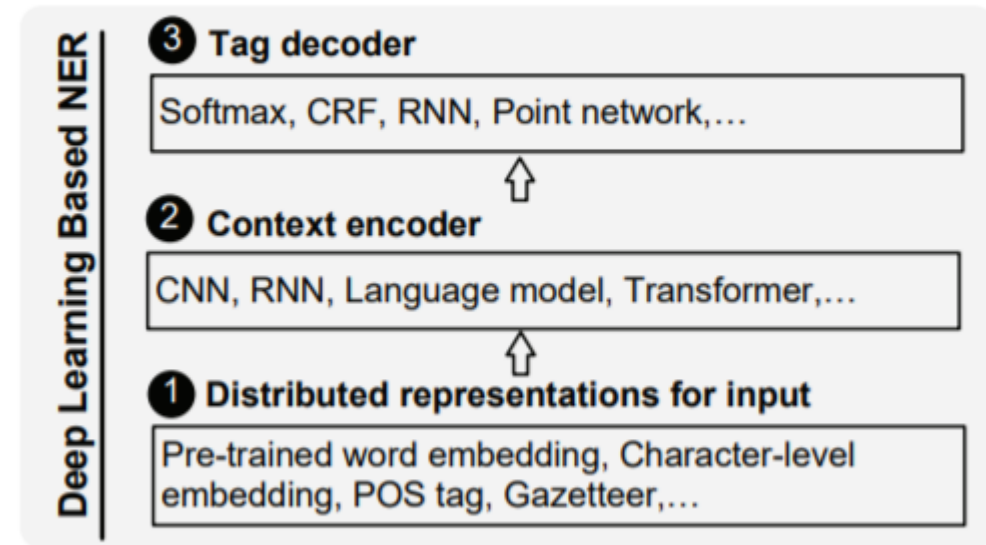
- In recent years, deep learning-based models are being used for building state-of-the-art systems for NER.
- There are many advantages of using DL techniques over the previously discussed approaches. Using the DL approach, the input data is mapped to a non-linear representation.
-

Methods of NER(Named Entity Recognition)

Deep Learning Approaches

- This approach helps to learn complex relations that are present in the input data.
- Another advantage is that we can avoid a lot of time and resources spent on feature engineering, which is required for the other traditional approaches.

B-PER I-PER E-PER O O O S-LOC O B-LOC E-LOC O
Michael Jeffrey Jordan was born in Brooklyn , New York .



Michael Jeffrey Jordan was born in Brooklyn, New York.

Analyzing text with NLTK



Chatbot using Dialogflow



Thank you



	Tag	Description	Example
Open Class	ADJ	Adjective: noun modifiers describing properties	<i>red, young, awesome</i>
	ADV	Adverb: verb modifiers of time, place, manner	<i>very, slowly, home, yesterday</i>
	NOUN	words for persons, places, things, etc.	<i>algorithm, cat, mango, beauty</i>
	VERB	words for actions and processes	<i>draw, provide, go</i>
	PROPN	Proper noun: name of a person, organization, place, etc..	<i>Regina, IBM, Colorado</i>
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	<i>oh, um, yes, hello</i>
Closed Class Words	ADP	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	<i>in, on, by, under</i>
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	<i>can, may, should, are</i>
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	<i>and, or, but</i>
	DET	Determiner: marks noun phrase properties	<i>a, an, the, this</i>
	NUM	Numeral	<i>one, two, 2026, 11:00, hundred</i>
	PART	Particle: a function word that must be associated with another word	<i>'s, not, (infinitive) to</i>
	PRON	Pronoun: a shorthand for referring to an entity or event	<i>she, who, I, others</i>
	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	<i>whether, because</i>
Other	PUNCT	Punctuation	<i>; , ()</i>
	SYM	Symbols like \$ or emoji	<i>\$, %</i>
	X	Other	<i>asdf, qwfg</i>